

HLD Case Study 8: Real-Time Matching & Recommendation (Tinder)

Key Focus Areas: Bidirectional mutual matching, real-time swipe ingestion, two-stage recommendation ranking, geospatial bucketing, and match notifications.

1. Problem Statement & Functional Requirements

Design the core discovery and mutual matching engine for a mobile dating application.

Requirements:

1. **Swipe Ingestion:** Users swipe Right (Like) or Left (Pass) at massive throughput ($> 50,000$ swipes/second).
 2. **Mutual Match Detection:** If both users swipe Right on each other, trigger a real-time **Mutual Match** event immediately.
 3. **Recommendation Feed:** Provide users with an endless stream of active, nearby profiles matching their preferences.
-

2. High-Level Architecture Diagram

flowchart TD

Client["User App"] → ALB["Application Load Balancer"]

ALB → Gateway["API Gateway"]

Gateway → SwipeSvc["Swipe Processing Service"]

SwipeSvc → MatchCache["Redis Cluster (Active Likes & Swipes)
Set: user_likes:A

SwipeSvc → Kafka["Kafka: swipe_events"]

Kafka → MatchWorker["Match Detection & Notification Worker"]

MatchWorker → MatchDB["Matches Database (Cassandra / DynamoDB)"]

MatchWorker → PushSvc["Push Notification Service (WebSockets)"]

Gateway → RecSvc["Recommendation Service"]

RecSvc → GeoIndex["Geospatial Profile Index (Uber H3 / Elasticsearch)"]

RecSvc → MLRanker["ML Candidate Ranking Model"]

3. Real-Time Mutual Match Detection Pipeline

How do we check if a swipe produces a mutual match in under 10 milliseconds without querying heavy relational databases?

```
sequenceDiagram
    autonumber
    actor A as User Alice
    participant Svc as Swipe Service
    participant Redis as Redis Cluster
    participant Kafka as Kafka Queue
    actor B as User Bob

    A->>Svc: swipe_right(target_user_id = Bob)
    Note over Svc: 1. Record Alice's like in Redis set
    Svc->>Redis: SADD user_likes:Alice Bob

    Note over Svc: 2. Atomic check: Did Bob already like Alice?
    Svc->>Redis: SISMEMBER user_likes:Bob Alice
    alt Mutual Match Found! (Bob already liked Alice)
        Redis->>Svc: True (Mutual Match!)
        Svc->>Kafka: emit_event("MATCH_CREATED", {Alice, Bob})
        Svc->>A: 200 OK: "It's a Match!"
        Kafka->>B: Push Notification: "You matched with Alice!"
    else No Match Yet
        Redis->>Svc: False
        Svc->>A: 200 OK: Swipe recorded
    end
end
```

4. The 2-Stage Recommendation Engine

Delivering a fresh deck of 20 profiles to a user involves two distinct stages:

```
flowchart LR
    AllProfiles["AllProfiles[\"Millions of Profiles\"]"] --> Stage1["Stage 1: Candidate Generation (Filter  
Stage1 --> Candidates[\"~1,000 Candidate Profiles\"]
    Candidates --> Stage2["Stage 2: ML Scoring & Ranking<br/>- Activity recency score<br/>
    Stage2 --> FinalDeck["Top 20 Ranked Profiles to User"]
```